

ST02 : Runtime Safety Filters for Humanoid Manipulation using Speed Limiting and Control Barrier Functions

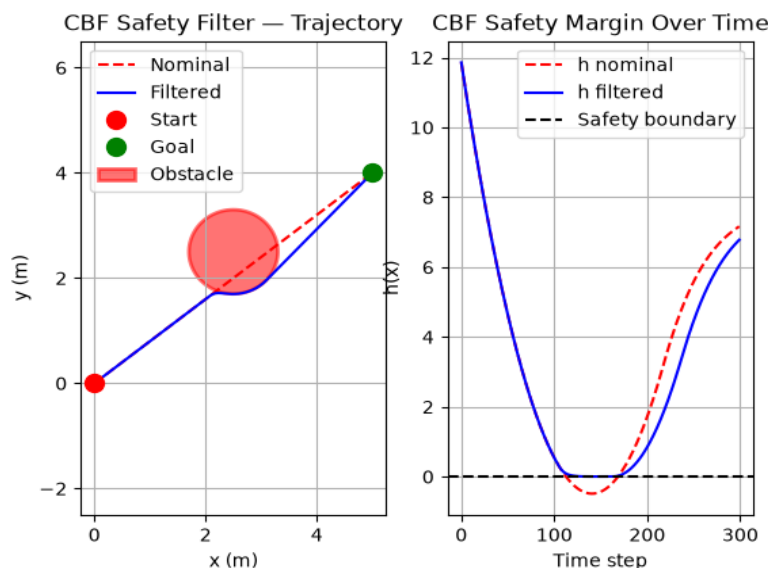
ST02 First Iteration:

Create a minimal Python prototype of a one-step safety filter for a simple reaching task.

The prototype may use a very simple 2D or 3D point-mass model instead of a full humanoid. A nominal command should move the point toward a target, while a circular or spherical obstacle defines an unsafe region. Implement at least one simple safety modification, for example distance-based speed limiting or a small quadratic program that minimally changes the nominal command.

The first iteration should show:

1. nominal motion without safety modification, filtered motion with the safety modification
2. a plot of the trajectory, obstacle, and minimum safety margin over time,



3. a short explanation of how this relates to humanoid safety filtering.
- The humanoid robot has 20+ degree of freedom and dozens of joints, and the motion of these end effectors can be represented using the point mass, as we can consider these point masses as the centre of mass. We can formulate our problem on the reduced order model using fewer constraints and we can simplify the problem by treating that point as a point mass and writing the safety constraint just for it, instead of writing a separate constraint for every joint. This gives us far fewer constraints to solve in real time, and it's the same idea used in real humanoid safety work, where end effector or centre of mass positions are tracked directly instead of the full joint state. Once the safety filter outputs a corrected velocity for that point, a whole-body controller maps it back down to the actual joint commands needed to move the real robot. We can consider the point mass as end effector of humanoid, the obstacles could be anything that can restrict the motion of humanoid like debris, walls, or its own limbs colliding with the torso.